



ΕΡΓΑΣΙΑ 3 – ΣΥΝΟΛΟΔΕΔΟΜΕΝΑ

ΜΥΕ041 - ΠΛΕ081: Διαχείριση Σύνθετων Δεδομένων

ΔΗΜΗΤΡΑ ΧΡΙΣΤΙΝΑ ΓΚΑΡΑΒΕΛΑ ΑΜ:5051

cs05051@uoi.gr

ΜΕΡΟΣ 1

Η πρώτη γραμμή ορίζει τη συνάρτηση `read_transactions(file_path: str) -> List[Set[int]]`. Με το `file_path: str` υποδεικνύουμε ότι η παράμετρος είναι μια συμβολοσειρά (η διαδρομή στο αρχείο), ενώ με το `-> List[Set[int]]` δηλώνουμε ότι η συνάρτηση επιστρέφει λίστα από σύνολα ακέραιων.

Δημιουργούμε μια κενή λίστα `transactions = []` που θα γεμίσουμε με ένα `set` για κάθε γραμμή του αρχείου. Χρησιμοποιούμε `set` για να αφαιρούνται αυτόματα τυχόν διπλότυπα αντικείμενα μέσα στην ίδια συναλλαγή. Χρησιμοποιούμε `with open(..., 'r')` για να ανοίξουμε με ασφάλεια το αρχείο (κλείνει αυτόματα στο τέλος). Για κάθε γραμμή κάνουμε `strip()` για να αφαιρέσουμε κενά και το `\n`. Αν η γραμμή είναι κενή, προσθέτουμε ένα κενό `set()` και συνεχίζουμε. Διαγράφουμε τα εξωτερικά [...] με `content = line[1:-1]`, κάνουμε `content.split(',')` για να χωρίσουμε στα κόμματα, και για κάθε μη-κενό κομμάτι μετατρέπουμε σε `int`, συλλέγοντας τα σε λίστα. Μετατρέπουμε τη λίστα `items` σε `set(items)` ώστε να αφαιρεθούν τυχόν διπλότυπα μέσα στην ίδια γραμμή, και το προσθέτουμε στη λίστα `transactions`. Όταν έχουμε διαβάσει όλες τις γραμμές, επιστρέφουμε τη λίστα `transactions`. Κάθε στοιχείο είναι ένα `set` που περιέχει όλα τα μοναδικά `item IDs` της αντίστοιχης γραμμής του αρχείου.

```
6 def read_transactions(file_path: str) -> List[Set[int]]:
7     transactions: List[Set[int]] = []
8     with open(file_path, 'r') as f:
9         for line in f:
10             line = line.strip()
11             if not line:
12                 transactions.append(set())
13                 continue
14             # remove [ ]
15             content = line[1:-1]
16             items = [int(x) for x in content.split(',') if x.strip()]
17             transactions.append(set(items))
18     return transactions
```

Με τον ίδιο τρόπο διαβάζουμε και το αρχείο `queries.txt`.

```
20 def read_queries(file_path: str) -> List[Set[int]]:
21     queries: List[Set[int]] = []
22     with open(file_path, 'r') as f:
23         for line in f:
24             line = line.strip()
25             if not line:
26                 queries.append(set())
27                 continue
28             # remove [ ]
29             content = line[1:-1]
30             items = [int(x) for x in content.split(',') if x.strip()]
31             queries.append(set(items))
32     return queries
```

Η `naive_containment_query` δέχεται δύο ορίσματα: από τη μία το `query`, που είναι ένα σύνολο (set) με τα IDs των αντικειμένων που ψάχνουμε, και από την άλλη το `transactions`, που είναι λίστα από σύνολα, όπου κάθε σύνολο απεικονίζει ποια αντικείμενα περιέχει η αντίστοιχη συναλλαγή.

Καθώς διασχίζουμε τη λίστα `transactions`, χρησιμοποιούμε τη συνάρτηση `enumerate`. Αυτή μας δίνει κάθε φορά δύο τιμές: πρώτον το `tid` (transaction ID), δηλαδή τη θέση της συναλλαγής στη λίστα, και δεύτερον το ίδιο το `items`, δηλαδή το σύνολο των αντικειμένων που περιέχει. Με αυτόν τον τρόπο, κρατάμε παράλληλα και τον δείκτη (ID) και το περιεχόμενο, χωρίς να χρειαστεί επιπλέον μετρήσεις ή αναζητήσεις. Για κάθε συναλλαγή, ελέγχουμε αν το σύνολο `query` είναι υποσύνολο (`issubset`) του `items`. Αυτό σημαίνει ότι ρωτάμε αν όλα τα αντικείμενα που ζητάει η ερώτηση υπάρχουν μέσα στη συγκεκριμένη συναλλαγή. Η μέθοδος `issubset` επιστρέφει `True` αν δεν λείπει κανένα ζητούμενο στοιχείο, ακριβώς αυτό που θέλουμε σε ένα `containment query` με `set semantics`. Κάθε φορά που ο έλεγχος `query.issubset(items)` βγάζει `True`, προσθέτουμε το `tid` (τον δείκτη της τρέχουσας γραμμής/συναλλαγής) σε μία λίστα `result`. Στο τέλος, όταν έχουμε περάσει όλες τις συναλλαγές, επιστρέφουμε αυτή τη λίστα με όλα τα IDs που ικανοποίησαν το ερώτημα. Με αυτόν τον τρόπο παίρνουμε άμεσα ποιοι πελάτες (IDs) αγόρασαν τουλάχιστον όλα τα αντικείμενα που ζητήσαμε.

```
36 def naive_containment_query(query: Set[int], transactions: List[Set[int]]) -> List[int]:
37     result = []
38     for tid, items in enumerate(transactions):
39         if query.issubset(items):
40             result.append(tid)
41     return result
```

Καθώς διασχίζουμε τη λίστα `transactions`, για κάθε μεμονωμένη συναλλαγή (που αντιπροσωπεύεται από το σύνολο `items`), ξεκινούμε με τη μεταβλητή `sig` να ισούται με το 0. Αυτή η τιμή λειτουργεί ως “καθαρός” bitmap, όπου αρχικά κανένα bit δεν είναι ενεργό. Στη συνέχεια, διατρέχουμε όλα τα `item` της τρέχουσας συναλλαγής. Για κάθε `item`, εκτελούμε τη λειτουργία `sig |= (1 << item)`, πρώτα δημιουργούμε με `1 << item` έναν αριθμό που έχει μόνο το bit `item` σε 1 (όλα τα υπόλοιπα 0) και μετά «ενώνουμε» με το τρέχον `sig` μέσω bitwise OR. Έτσι, ενεργοποιούμε μόνο το bit που αντιστοιχεί σε αυτό το `item`, χωρίς να πειράζουμε τα υπόλοιπα bits που μπορεί να έχουν ήδη τεθεί από προηγούμενα αντικείμενα.

Μετά την επανάληψη όλων των `item`, ο `sig` αντιπροσωπεύει πλήρως τη συνδυαστική εμφάνιση των αντικειμένων σε αυτήν τη συναλλαγή. Τον προσθέτουμε στη λίστα `sigfile`. Μόλις περάσουμε όλες τις συναλλαγές, η `sigfile` είναι μία λίστα ακεραίων, όπου ο i -οστός ακέραιος είναι το bitmap της i -οστής συναλλαγής.

Τέλος, ανοίγουμε το αρχείο sigfile.txt για εγγραφή και για κάθε sig της λίστας γράφουμε μία γραμμή που περιέχει απλώς τον ακέραιο. Έτσι, η 1η γραμμή είναι η υπογραφή της συναλλαγής id=0, η 2η η υπογραφή για id=1 κ.ο.κ. Αυτό το αρχείο μας επιτρέπει να φορτώσουμε ξανά γρήγορα τους bitmaps χωρίς να ξαναδιαβάσουμε ολόκληρο το αρχείο των συναλλαγών.

```
44 def build_signature_file(transactions: List[Set[int]]) -> List[int]:
45     sigfile = []
46     for items in transactions:
47         sig = 0
48         for item in items:
49             if item < 0:
50                 raise ValueError(f"Invalid item id: {item}")
51             sig |= (1 << item)
52         sigfile.append(sig)
53     with open('sigfile.txt', 'w') as fout:
54         for sig in sigfile:
55             fout.write(str(sig) + '\n')
56     return sigfile
```

Ακριβώς όπως κάναμε για τις συναλλαγές, ξεκινάμε με έναν ακέραιο qsig = 0. Για κάθε item που ζητάει η ερώτηση (query), εκτελούμε qsig |= (1 << item). Έτσι δημιουργείται ένας αριθμός-bitmap όπου κάθε bit στη θέση *i* (αντικείμενο *i*) είναι 1 μόνον αν το αντικείμενο *i* είναι στο query. Αυτό μας επιτρέπει να συγκρίνουμε το ερώτημα με κάθε συναλλαγή μέσω γρήγορων bitwise πράξεων.

Για κάθε αποθηκευμένο sig (bitmap) μιας συναλλαγής, κάνουμε τη δοκιμή (sig & qsig) == qsig, το sig & qsig κρατάει μόνο τα bits που είναι 1 και στα δύο. Αν το αποτέλεσμα ταυτίζεται με το qsig, τότε όλα τα bits που θέλει η ερώτηση είναι παρόντα (και άρα ενεργά) στη συγκεκριμένη συναλλαγή. Όποτε ο παραπάνω έλεγχος βγάζει True, προσθέτουμε το tid (τον δείκτη της τρέχουσας συναλλαγής) στη λίστα των αποτελεσμάτων. Στο τέλος επιστρέφουμε αυτή τη λίστα με τα tid όλων των συναλλαγών που περιέχουν όλα τα αντικείμενα της ερώτησης. Έτσι, αποφεύγουμε επαναλαμβανόμενο set-subset έλεγχο.

```
59 def signature_containment_query(query: Set[int], sigfile: List[int]) -> List[int]:
60     qsig = 0
61     for item in query:
62         qsig |= (1 << item)
63     result = []
64     for tid, sig in enumerate(sigfile):
65         if (sig & qsig) == qsig:
66             result.append(tid)
67     return result
```

Δημιουργούμε μια δομή όπου το κλειδί είναι το item ID και η τιμή είναι ένας ακέραιος (int) που λειτουργεί ως bitmap: κάθε bit του αντιστοιχεί σε μία μοναδική συναλλαγή, με το LSB (bit 0) να αντιστοιχεί στην πρώτη συναλλαγή (tid = 0), το bit 1 στη δεύτερη (tid = 1) κ.ο.κ.

Διασχίζουμε όλες τις συναλλαγές με `enumerate`, παίρνοντας το `tid` και το σύνολο των `items`. Για κάθε `item` μέσα σε αυτήν τη συναλλαγή, κάνουμε: `bitslice[item] |= (1 << tid)`, το `1 << tid`, δημιουργούμε έναν ακέραιο με μόνο το bit `tid` σε 1, και το OR (`|=`) το ενώνει με τον υπάρχοντα bitmap. Έτσι, κάθε φορά που ένα αντικείμενο εμφανίζεται σε μια συγκεκριμένη συναλλαγή, το αντίστοιχο bit θέτει value 1. Μόλις ολοκληρώσουμε τη διαδικασία για όλα τα `items`, γράφουμε σε `bitslice.txt` κάθε γραμμή με τη μορφή: `item: bitmap`, όπου `item` είναι το ID του αντικειμένου και `bitmap` ο ακέραιος που σχηματίζεται.

```
70 def build_bitslice_file(transactions: List[Set[int]]) -> Dict[int, int]:
71     bitslice: Dict[int, int] = {}
72     for tid, items in enumerate(transactions):
73         for item in items:
74             if item < 0:
75                 raise ValueError(f"Invalid item id: {item}")
76             bitslice.setdefault(item, 0)
77             bitslice[item] |= (1 << tid)
78     with open('bitslice.txt', 'w') as fout:
79         for item in sorted(bitslice.keys()):
80             fout.write(f"{item}: {bitslice[item]}\n")
81     return bitslice
```

Ξεκινάμε παίρνοντας το bitmap (`bitslice[item]`) του πρώτου `item` που ζητάει η ερώτηση και το αποθηκεύουμε σε μία μεταβλητή `acc`. Έτσι ο `acc` περιέχει αρχικά όλες τις συναλλαγές όπου εμφανίζεται αυτό το πρώτο αντικείμενο. Για κάθε επόμενο `item` στο `query`, παίρνουμε το αντίστοιχο bitmap και εκτελούμε `acc &= bitslice[item]`. Με αυτή τη bitwise AND πράξη κρατάμε μόνο τα bits (δηλ. τις συναλλαγές) που είναι ενεργά και στο `acc` και στο bitmap του τρέχοντος `item`. Έτσι στο τέλος, ο `acc` αντιπροσωπεύει μόνο τις συναλλαγές που περιέχουν όλα τα αντικείμενα του `query`. Όσο υπάρχουν ακόμη bits να επεξεργαστούμε (`while acc:`), βρίσκουμε το χαμηλότερο ενεργό bit με `lsb = acc & -acc`. Το `lsb` είναι μια δύναμη του δύο που δείχνει πού βρίσκεται το ελάχιστο set bit. Με `lsb.bit_length() - 1` υπολογίζουμε σε ποιο `tid` αντιστοιχεί. Προσθέτουμε αυτό το `tid` στη λίστα `result` και μετά σβήνουμε το bit από το `acc` με `acc &= acc - 1`. Επαναλαμβάνουμε μέχρι να μην μένουν bits, οπότε στο `result` έχουμε όλα τα IDs των κοινών συναλλαγών.

```
84 def bitslice_containment_query(query: Set[int], bitslice: Dict[int, int]) -> List[int]:
85     it = iter(query)
86     try:
87         first = next(it)
88     except StopIteration:
89         return []
90     if first not in bitslice:
91         return []
92     acc = bitslice[first]
93     for item in it:
94         if item not in bitslice:
95             return []
96         acc &= bitslice[item]
97     result = []
98     while acc:
99         lsb = (acc & -acc)
100         tid = lsb.bit_length() - 1
101         result.append(tid)
102         acc &= acc - 1
103     result.sort()
104     return result
```

Για κάθε item που συναντάμε σε κάθε συναλλαγή, επιλέγουμε το dict inv ώστε το inv[item] να είναι μια λίστα με όλα τα tid (transaction IDs) όπου εμφανίζεται αυτό το αντικείμενο. Δηλαδή, διασχίζοντας με enumerate(transactions) παίρνουμε το tid και το items σύνολο, και για κάθε item μέσα σε αυτό προσθέτουμε το tid στην αντίστοιχη λίστα: inv.setdefault(item, []).append(tid). Έτσι, στο τέλος η δομή inv χαρτογραφεί κάθε αντικείμενο στις συναλλαγές όπου συμμετέχει.

Μόλις συμπληρώσουμε όλες τις inverted lists, διασχίζουμε κάθε λίστα inv[item] και την ταξινομούμε σε αύξουσα σειρά (lst.sort()). Αυτό διασφαλίζει ότι οι tid εμφανίζονται με μικρότερο προς μεγαλύτερο, διευκολύνοντας μετέπειτα κατανεμημένες ή διαδοχικές τομές. Τέλος, γράφουμε σε invfile.txt κάθε γραμμή ως: item: [tid1, tid2, tid3, ...] ώστε να έχουμε ένα έτοιμο inverted file όπου το κλειδί είναι το αντικείμενο και η τιμή η ταξινομημένη λίστα συναλλαγών.

```
107 def build_inverted_file(transactions: List[Set[int]]) -> Dict[int, List[int]]:
108     inv: Dict[int, List[int]] = {}
109     for tid, items in enumerate(transactions):
110         for item in items:
111             inv.setdefault(item, []).append(tid)
112     for lst in inv.values():
113         lst.sort()
114     with open('invfile.txt', 'w') as fout:
115         for item in sorted(inv.keys()):
116             fout.write(f"{item}: {inv[item]}\n")
117     return inv
```

Η intersect_sorted παίρνει δύο λίστες ακεραίων a και b που είναι ήδη ταξινομημένες σε αύξουσα σειρά. Ο στόχος είναι να βρούμε την τομή τους, δηλαδή τα στοιχεία που υπάρχουν και στις δύο λίστες, διασχίζοντάς τες με μία μόνο διπλή επανάληψη. Δημιουργούμε μία κενή λίστα result που θα συγκεντρώσει τα κοινά στοιχεία. Ορίζουμε δύο δείκτες i και j, αρχικά στο 0, για να περιηγηθούμε παράλληλα στις λίστες a και b αντίστοιχα.

Μέχρι να φτάσουμε στο τέλος μίας από τις δύο λίστες (while i < len(a) and j < len(b):), συγκρίνουμε τα τρέχοντα στοιχεία a[i] και b[j].

- Αν είναι ίσα, έχουμε κοινό στοιχείο: το προσθέτουμε στο result και αυξάνουμε και τους δύο δείκτες (i += 1; j += 1).
- Αν a[i] < b[j], σημαίνει ότι το a[i] δεν θα ταιριάξει με κανένα μελλοντικό στοιχείο της b (όλα τα επόμενα της b θα είναι ακόμα μεγαλύτερα), οπότε αυξάνουμε μόνο τον i.
- Διαφορετικά (a[i] > b[j]), αυξάνουμε μόνο τον j.

Όταν ο βρόχος τερματίσει (έχουμε φτάσει στο τέλος μίας λίστας), το result περιέχει όλα τα κοινά στοιχεία με τη σωστή ταξινόμηση. Το επιστρέφουμε για χρήση στις inverted-list τομές.

```
120 def intersect_sorted(a: List[int], b: List[int]) -> List[int]:
121     result = []
122     i = j = 0
123     while i < len(a) and j < len(b):
124         if a[i] == b[j]:
125             result.append(a[i])
126             i += 1; j += 1
127         elif a[i] < b[j]:
128             i += 1
129         else:
130             j += 1
131     return result
```

Αρχικά μετατρέπουμε το query από σετ σε λίστα (items = list(query)) ώστε να έχουμε σταθερή σειρά επανάληψης και να μπορούμε να πάρουμε καθαρά το πρώτο στοιχείο. Αν η λίστα items είναι κενή, επιστρέφουμε αμέσως κενό αποτέλεσμα, αφού δεν υπάρχει τίποτα να ελέγξουμε. Στη συνέχεια ελέγχουμε αν το πρώτο item υπάρχει στο inv, αν όχι, καμία συναλλαγή δεν το περιέχει, οπότε και η τελική τομή θα είναι κενή. Αν υπάρχει, θέτουμε ως αρχική λίστα res την inverted-list αυτού του πρώτου item, που περιέχει όλα τα tid όπου εμφανίζεται. Για κάθε επόμενο item του query ελέγχουμε ξανά την ύπαρξή του στο inv. Αν λείπει, τερματίζουμε με κενό, διαφορετικά υπολογίζουμε τη διαδοχική τομή με intersect_sorted(res, inv[item]), κρατώντας μόνο τα κοινά tid. Αν σε κάποιο σημείο η res γίνει κενή, σταματάμε νωρίς γιατί δεν υπάρχουν πλέον κοινές συναλλαγές. Στο τέλος, η res περιέχει όλους τους tid που βρίσκονται σε κάθε inverted-list των ζητούμενων αντικειμένων, δηλαδή τις συναλλαγές που ικανοποιούν πλήρως το containment query. Παίρνει τις inverted-lists κάθε item της ερώτησης και υπολογίζει διαδοχικές τομές, επιστρέφοντας τα κοινά tid.

```
134 def inverted_containment_query(query: Set[int], inv: Dict[int, List[int]]) -> List[int]:
135     items = list(query)
136     if not items:
137         return []
138     if items[0] not in inv:
139         return []
140     res = inv[items[0]]
141     for item in items[1:]:
142         if item not in inv:
143             return []
144         res = intersect_sorted(res, inv[item])
145         if not res:
146             break
147     return res
```

Αρχικά βεβαιωνόμαστε ότι ο χρήστης πέρασε ακριβώς τέσσερα ορίσματα (εκτός του ονόματος του script) με τον έλεγχο `len(sys.argv) != 5`: αν η κλήση δεν είναι έγκυρη τυπώνουμε “Usage: ...” και τερματίζουμε με `sys.exit(1)`. Στη συνέχεια αναθέτουμε τα ορίσματα σε `tfile` (αρχείο transactions), `qfile` (αρχείο queries), `qnum_str` (ID ερώτησης ή “-1” για όλες) και `method_str` (κωδικός μεθόδου 0–3 ή -1 για όλες), μετατρέποντας με `int()` τα δύο τελευταία σε `qnum` και `method`. Φορτώνουμε όλα τα δεδομένα στη μνήμη μέσω `read_transactions(tfile)` και `read_queries(qfile)`, χτίζουμε μίας φορές τους δείκτες signature file (`sigfile`), bitslice index (`bitslice`) και inverted index (`invfile`) και γράφουμε παράλληλα τα αντίστοιχα αρχεία `sigfile.txt`, `bitslice.txt`, `invfile.txt`. Με το dict `names` χαρτογραφούμε κάθε κωδικό σε ανθρώπινο όνομα μεθόδου, και με το `funcs` τον συνδέουμε στην κατάλληλη συνάρτηση containment-query. Αν `method != -1` τρέχουμε μόνο τη ζητούμενη μέθοδο, αλλιώς βάζουμε σειρά εκτέλεσης όλων [0,1,2,3]. Για κάθε επιλεγμένη μέθοδο παίρνουμε `name` και `func`, και αν `qnum == -1` μετράμε απλώς σε loop τον συνολικό χρόνο εκτέλεσης σε όλες τις ερωτήσεις, ενώ αν `qnum ≥ 0` εκτελούμε τη μέθοδο μόνο στη `queries[qnum]`—μετράμε `time.time()` πριν/μετά, εκτυπώνουμε το αποτέλεσμα ως `set(res)` για ευανάγνωστη μορφή {...} και στη συνέχεια τον υπολογισμένο χρόνο.

```
150 def main():
151     if len(sys.argv) != 5:
152         print("Usage: python program.py <transactions file> <queries file> <qnum> <method>")
153         sys.exit(1)
154     tfile, qfile, qnum_str, method_str = sys.argv[1:]
155     qnum = int(qnum_str)
156     method = int(method_str)
157
158     transactions = read_transactions(tfile)
159     queries = read_queries(qfile)
160
161     sigfile = build_signature_file(transactions)
162     bitslice = build_bitslice_file(transactions)
163     invfile = build_inverted_file(transactions)
164
165     names = {0: 'Naive Method', 1: 'Signature File', 2: 'Bitsliced Signature File', 3: 'Inverted File'}
166     funcs = {
167         0: lambda q: naive_containment_query(q, transactions),
168         1: lambda q: signature_containment_query(q, sigfile),
169         2: lambda q: bitslice_containment_query(q, bitslice),
170         3: lambda q: inverted_containment_query(q, invfile),
171     }
172
173     methods = [method] if method != -1 else [0,1,2,3]
174
175     for m in methods:
176         name = names[m]
177         func = funcs[m]
178         if qnum == -1:
179             start = time.time()
180             for q in queries:
181                 _ = func(q)
182             elapsed = time.time() - start
183             print(f"{name} Computation time = {elapsed:.6f}")
184         else:
185             query = queries[qnum]
186             start = time.time()
187             res = func(query)
188             elapsed = time.time() - start
189             print(f"{name} result:")
190             print(set(res))
191             print(f"{name} Computation time = {elapsed:.6f}")
192
```

ΜΕΡΟΣ 2

Ξεκινάμε με τη συνάρτηση `read_transactions(file_path)`, που παίρνει ως είσοδο το όνομα ή τη διαδρομή του αρχείου των συναλλαγών. Δημιουργούμε μία κενή λίστα `transactions` στην οποία θα προσθέτουμε, για κάθε γραμμή του αρχείου, τη λίστα των αντικειμένων που περιέχει. Χρησιμοποιούμε την `with open()` ώστε να ανοίξουμε το αρχείο σε `read mode` και να φροντίσουμε αυτόματα για το κλείσιμό του στο τέλος, ακόμη κι αν συμβεί σφάλμα. Διαβάζουμε κάθε γραμμή με `for line in f`, κάνουμε `line.strip()` για να αφαιρέσουμε κενά, tabs ή το `\n`. Αν μετά το `strip` η γραμμή είναι κενή, απλώς τη σκανάρουμε με `continue`. Με `ast.literal_eval(line)` ερμηνεύουμε το κείμενο "[1,2,3]" ως πραγματική Python λίστα `items`. Εφαρμόζουμε `[str(i) for i in items]` ώστε κάθε στοιχείο της λίστας να μετατραπεί σε `string`. Έτσι διασφαλίζουμε ενιαίο τύπο (`strings`) σε όλες τις εγγραφές. Προσθέτουμε τη νέα λίστα από `strings` στη `transactions`. Μόλις τελειώσουμε με όλες τις γραμμές, επιστρέφουμε τη λίστα `transactions`, όπου κάθε εγγραφή είναι λίστα IDs σε μορφή `string`.

```
6 def read_transactions(file_path):
7     transactions = []
8     with open(file_path, 'r') as f:
9         for line in f:
10             line = line.strip()
11             if not line:
12                 continue
13             items = ast.literal_eval(line)
14             transactions.append([str(i) for i in items])
15     return transactions
```

Η `build_inverted_index` ξεκινάει από τη βασική ιδέα ενός `inverted index`: για κάθε `item` θέλουμε να έχουμε γρήγορη πρόσβαση στις συναλλαγές (`tid`) όπου εμφανίζεται. Χρησιμοποιούμε τη δομή `inverted[item][tid]` για να μετράμε πόσες φορές το ίδιο `item` εμφανίζεται μέσα σε κάθε συγκεκριμένη συναλλαγή. Έτσι, αν ένα αντικείμενο υπάρχει τρεις φορές στην ίδια γραμμή (`bag semantics`), θα αποθηκευτεί `occ = 3`. Παράλληλα, η `trf[item]` μετράει σε πόσες διαφορετικές συναλλαγές εμφανίζεται το `item`. Για να αποφύγουμε διπλότυπα μέσα στην ίδια συναλλαγή, κρατάμε ένα προσωρινό `set seen`, κάθε φορά που βλέπουμε για πρώτη φορά ένα `item` σε μια συναλλαγή, αυξάνουμε το `trf[item]` κατά 1. Η μεταβλητή `T = len(transactions)` αποθηκεύει απλώς πόσες γραμμές (συναλλαγές) διαβάσαμε συνολικά, και χρησιμοποιείται μετά για τον υπολογισμό των βαρών. Για κάθε `item` υπολογίζουμε `weight = T / trf[item]`. Μόλις ολοκληρώσουμε τις μετρήσεις, φτιάχνουμε για κάθε `item` τη λίστα ζευγών `[tid, occurrences]` (από το εσωτερικό `dict`) και την ταξινομούμε κατά `tid` ώστε οι συναλλαγές να είναι σε αύξουσα σειρά. Η συνάρτηση επιστρέφει τρία στοιχεία: **inv_lists** (το `inverted index`, `item` → `sorted [[tid, occ], ...]`), **weights** (το βάρος για κάθε `item`) και **T** (ο συνολικός αριθμός των συναλλαγών).

```

17 def build_inverted_index(transactions):
18     inverted = defaultdict(lambda: defaultdict(int)) # item -> {tid: occ}
19     trf = defaultdict(int) # item -> number of transactions containing item
20     T = len(transactions)
21     for tid, items in enumerate(transactions):
22         seen = set()
23         for item in items:
24             inverted[item][tid] += 1
25             if item not in seen:
26                 trf[item] += 1
27                 seen.add(item)
28     weights = {item: T / freq for item, freq in trf.items()}
29     inv_lists = {
30         item: sorted([[tid, occ] for tid, occ in tids.items()], key=lambda x: x[0])
31         for item, tids in inverted.items()
32     }
33     return inv_lists, weights, T

```

Η `write_invfile(inv_lists, weights, output_path)` παίρνει τρεις παραμέτρους: **inv_lists** (το inverted index, ένα dict που χαρτογραφεί κάθε item σε λίστα από [tid, occ]), **weights** (dict όπου κάθε item αντιστοιχεί στο βάρος του (π.χ. $T/\text{trf}[\text{item}]$)) και **output_path** (τη διαδρομή του αρχείου στο οποίο θέλουμε να γράψουμε τα δεδομένα).

Με την εντολή `with open(output_path, 'w') as f`: ανοίγουμε το αρχείο σε write mode ('w'). Η χρήση του `with` εξασφαλίζει ότι, ακόμη και αν υπάρξει σφάλμα μέσα στο μπλοκ, το αρχείο θα κλείσει σωστά στο τέλος. Στη συνέχεια παίρνουμε όλα τα item από τα `inv_lists.keys()` και τα ταξινομούμε με: `sorted(inv_lists.keys(), key=lambda x: int(x) if x.isdigit() else x)`. Έτσι, αν ένα item είναι καθαρά αριθμητικό string ("10"), θα μετατραπεί σε `int(10)` για αριθμητική ταξινόμηση, ενώ διαφορετικά παραμένει string και ταξινομείται λεξικογραφικά. Αυτή η λογική εξασφαλίζει ότι π.χ. το "10" έρχεται μετά το "2", όπως σε έναν αυστηρά αριθμητικό κατάλογο. Για κάθε ταξινομημένο item γράφουμε μια γραμμή: `{item}: {weights[item]}, {inv_lists[item]}`, όπου **weights[item]** είναι το βάρος του αντικειμένου και **inv_lists[item]** η αντίστοιχη inverted list [tid, occurrences]. Το `\n` στο τέλος κάθε γραμμής δημιουργεί το νέο line break. Με αυτό τον τρόπο παράγουμε ένα ευανάγνωστο αρχείο όπου κάθε γραμμή περιέχει, με σταθερή σειρά, το item, το βάρος του και τις συναλλαγές όπου εμφανίστηκε μαζί με τις μετρήσεις τους.

```

35 def write_invfile(inv_lists, weights, output_path):
36     with open(output_path, 'w') as f:
37         for item in sorted(inv_lists.keys(), key=lambda x: int(x) if x.isdigit() else x):
38             f.write(f"{item}: {weights[item]}, {inv_lists[item]}\n")

```

Όπως το `read_transactions` αλλά για το αρχείο `queries.txt`.

```
40 def read_queries(file_path):
41     queries = []
42     with open(file_path, 'r') as f:
43         for line in f:
44             line = line.strip()
45             if not line:
46                 continue
47             items = ast.literal_eval(line)
48             queries.append([str(i) for i in items])
49     return queries
```

Η `naive_method(query, transactions, weights, k)` παίρνει τέσσερις παραμέτρους: **query** (το σύνολο με τα ζητούμενα item ids), **transactions** (η λίστα που αναπαριστά κάθε συναλλαγή ως λίστα ή set αντικειμένων), **weights** (ένα dict που χαρτογραφεί κάθε item στο βάρος του) και **k** (ο αριθμός των κορυφαίων απαντήσεων που θέλουμε να επιστρέψουμε). Ξεκινάμε με μία κενή λίστα `rels` όπου θα μαζέψουμε τα ζεύγη `[score, tid]`.

Με `enumerate(transactions)` παίρνουμε για κάθε συναλλαγή το `tid` (ID) και το `items`.

Για κάθε συναλλαγή δημιουργούμε ένα `defaultdict(int)` με όνομα `count`, που θα καταμετρά πόσες φορές εμφανίζεται κάθε ζητούμενο item μέσα στη συναλλαγή, αυτό υλοποιεί `bag semantics`. Διατρέχουμε όλα τα `i` μέσα στο `items`: όποτε `i` in `query`, αυξάνουμε `count[i] += 1`.

Έτσι, αν ένα item επαναλαμβάνεται, συνεισφέρει πολλαπλά στον τελικό υπολογισμό.

Για κάθε συναλλαγή, φτιάχνουμε το `score rel` ως άθροισμα για κάθε ζητούμενο `i`: `rel += count[i] * weights.get(i, 0)`. Το `weights.get(i, 0)` εξασφαλίζει πως αν κάποιο `i` δεν έχει βάρος, παίρνει 0. Έτσι οι συχνότερες και σημαντικότερες εμφανίσεις αυξάνουν περισσότερο το `rel`.

Αν `rel > 0` (υπήρξε τουλάχιστον ένα κοινό item), προσθέτουμε `[rel, tid]` στη `rels`.

Μετά ταξινομούμε τη `rels` φθίνουσα με `key` το `rel` (το πρώτο στοιχείο των υπο-λιστών), ώστε πρώτα να έρχονται οι υψηλότερες τιμές. Τέλος, παίρνουμε και επιστρέφουμε τα πρώτα `k` στοιχεία της ταξινομημένης λίστας `rels[:k]`, δηλαδή τις `k` πιο σχετικές συναλλαγές με τα αντίστοιχα `scores`.

```
51 def naive_method(query, transactions, weights, k):
52     rels = []
53     for tid, items in enumerate(transactions):
54         count = defaultdict(int)
55         for i in items:
56             if i in query:
57                 count[i] += 1
58         rel = sum(count[i] * weights.get(i, 0) for i in query)
59         if rel > 0:
60             rels.append([rel, tid])
61     rels.sort(key=lambda x: x[0], reverse=True)
62     return rels[:k]
```

Η `inverted_method(query, inv_lists, weights, k)` υπολογίζει τα k κορυφαία relevance scores χρησιμοποιώντας τον inverted index και προκαθορισμένα βάρη. Πρώτα φτιάχνουμε ένα `defaultdict(float)` (`rel_dict`) για να συγκεντρώσουμε τα scores ανά συναλλαγή (`tid`). Στη συνέχεια, για κάθε `item` στο ερώτημα, αν υπάρχει στο `inv_lists`, παίρνουμε το βάρος $w = \text{weights.get}(\text{item}, 0)$ και διατρέχουμε τη λίστα `[tid, occ]` του `item`, αυξάνοντας το `rel_dict[tid] += occ * w`, έτσι κάθε επανεμφάνιση συνεισφέρει στο σκορ ανάλογα με το πόσες φορές εμφανίζεται και πόσο σημαντικό είναι. Μόλις ολοκληρωθούν όλα τα `contributions`, φτιάχνουμε τη λίστα `rels = [[rel, tid] for tid, rel in rel_dict.items() if rel > 0]`, την ταξινομούμε φθίνουσα με βάση το `rel` (πρώτο στοιχείο κάθε ζεύγους) και επιστρέφουμε τα πρώτα k αποτελέσματα, δηλαδή τις k συναλλαγές με τα υψηλότερα relevance scores.

```
64 def inverted_method(query, inv_lists, weights, k):
65     rel_dict = defaultdict(float)
66     for i in query:
67         if i not in inv_lists:
68             continue
69         w = weights.get(i, 0)
70         for tid, occ in inv_lists[i]:
71             rel_dict[tid] += occ * w
72     rels = [[rel, tid] for tid, rel in rel_dict.items() if rel > 0]
73     rels.sort(key=lambda x: x[0], reverse=True)
74     return rels[:k]
```

Πρώτα ελέγχουμε ότι ο χρήστης έδωσε ακριβώς πέντε ορίσματα (εκτός του ονόματος του script) — σε διαφορετική περίπτωση τυπώνουμε ένα σύντομο μήνυμα χρήσης και τερματίζουμε, διασφαλίζοντας έτσι σωστή κλήση. Στη συνέχεια αναθέτουμε τα ορίσματα σε `tx_file` (το αρχείο των συναλλαγών), `q_file` (το αρχείο των ερωτημάτων), `qnum` (το ID ερώτησης ή `-1` για όλες), `method` (`0 = Naïve`, `1 = Inverted`, `-1 =` και οι δύο) και `k` (πόσες κορυφαίες απαντήσεις). Κάνουμε `int()` σε `qnum`, `method` και `k` ώστε να μπορούμε να τα χρησιμοποιήσουμε αριθμητικά. Έπειτα φορτώνουμε όλες τις συναλλαγές με `read_transactions`, χτίζουμε τον inverted index (`inv_lists, weights`) με `build_inverted_index` και γράφουμε το `invfileocc.txt` για έλεγχο με `write_invfile`. Διαβάζουμε τα queries με `read_queries`. Αν η μέθοδος είναι `Naïve` (`0` ή `-1`), τρέχουμε τη `naïve` μεθοδολογία είτε σε όλες τις ερωτήσεις (χωρίς εκτύπωση, για μέτρηση χρόνου) είτε σε μία συγκεκριμένη (`queries[qnum]`, με εκτύπωση αποτελέσματος), μετρώντας τον χρόνο πριν και μετά. Το ίδιο κάνουμε και για την `inverted-list` μέθοδο (`1` ή `-1`): είτε όλες τις ερωτήσεις χωρίς εκτύπωση είτε μία με εκτύπωση, πάντα με χρονισμό `time.time()`.

```

82 def main():
83     parser = argparse.ArgumentParser(description='Relevance queries methods')
84     parser.add_argument('transactions', help='Transactions file')
85     parser.add_argument('queries', help='Queries file')
86     parser.add_argument('qnum', type=int, help='Query id or -1 for all')
87     parser.add_argument('method', type=int, help='Method: -1 all, 0 naive, 1 inverted')
88     parser.add_argument('k', type=int, help='Top-k results')
89     args = parser.parse_args()
90
91     transactions = read_transactions(args.transactions)
92     queries = read_queries(args.queries)
93
94     inv, idf, T = build_invfileocc(transactions, 'invfileocc.txt')
95
96     def run_naive(q):
97         start = time.perf_counter()
98         res = relevance_naive(transactions, queries, idf, T, q)
99         elapsed = time.perf_counter() - start
100         return res, elapsed
101
102     def run_inv(q):
103         start = time.perf_counter()
104         res = relevance_inverted(inv, idf, T, q)
105         elapsed = time.perf_counter() - start
106         return res, elapsed
107
108     methods = []
109     if args.method in (-1, 0): methods.append(('Naive Method', run_naive))
110     if args.method in (-1, 1): methods.append(('Inverted File Method', run_inv))
111
112     q_indices = [args.qnum] if args.qnum != -1 else list(range(len(queries)))
113
114     for name, runner in methods:
115         total_time = 0.0
116         results = None
117         for qi in q_indices:
118             res, t = runner(queries[qi])
119             total_time += t
120             if len(q_indices) == 1:
121                 results = res[:args.k]
122         if len(q_indices) == 1:
123             print(f'{name} result:')
124             print(results)
125     print(f'{name} computation time = {total_time:.6f} seconds")

```